Lecture 4: Trees and Binary Trees

CSI2110 – Data Structures and Algorithms

Lecture study notes

Source document: `../lec04-treeandbinarytree.pdf`. These notes synthesize the source and add worked explanations. References use PDF page numbers. Pseudocode includes boundary cases.

Contents

1 Graphs, trees, and rooted terminology **2**

2 Binary-tree variants **2**

3 Counting and height properties **2**

3.1 Any tree . . . 2

3.2 Any binary tree . . . 3

3.3 Full binary tree . . . 3

3.4 Perfect and complete trees . . . 3

4 Tree ADTs **3**

5 Traversal orders **4**

5.1 Breadth-first traversal . . . 4

5.2 Recursive depth-first traversal . . . 4

5.3 Iterative preorder and Euler tour . . . 5

6 Traversal applications **5**

6.1 Structured documents and directory sizes . . . 5

6.2 Subtree height . . . 5

6.3 Expression trees . . . 5

7 Representations and updates **6**

7.1 Linked binary tree . . . 6

7.2 Array binary tree . . . 6

7.3 General tree and binary encoding . . . 7

1 Graphs, trees, and rooted terminology

Source: PDF pages 5–11. A graph $G = (V, E)$ has vertices and edges. An undirected edge joins two vertices. A path follows adjacent vertices; a simple path does not repeat vertices. A cycle closes a path without otherwise repeating vertices. A connected graph has a path between every pair of vertices.

A **tree** is a connected, acyclic undirected graph. Selecting a root determines parent/child relationships. The root has no parent; every other node has exactly one parent.

- Siblings share a parent; ancestors are parents, grandparents, etc.; descendants are children, grandchildren, etc.
- An internal node has at least one child; an external node (leaf) has none.
- The subtree rooted at v contains v and all its descendants.
- Distance counts edges. The root has depth zero. A node's depth is its distance from the root.
- A nonempty tree's height is its maximum node depth, or its longest root-to-leaf path length in edges. A one-node tree has height zero.

2 Binary-tree variants

Source: PDF pages 13–17.

Type	Condition
Binary	At most two children per node, distinguished as left and right.
Full (proper)	Every node has either zero or two children.
Perfect	Full, with all leaves at the same depth.
Complete	All levels before the last are full; the last is filled left to right.

A perfect tree is both full and complete. A full tree need not be complete or balanced. A complete tree may have one node with only a left child. Decision trees and expression trees with binary operators are natural examples of full trees.

3 Counting and height properties

Source: PDF pages 19–33. Let n be total nodes, n_I internal nodes, n_E leaves, and h height. Assume nonempty trees and base-two logarithms.

3.1 Any tree

There is exactly one *simple* path between every pair of nodes. Connectivity gives existence; two distinct simple paths would imply a cycle. A tree has $n-1$ edges because every non-root node contributes one parent edge.

3.2 Any binary tree

At level i there are at most 2^i nodes. Therefore,

$$\begin{aligned} h+1 \le n &\le 2^{h+1}-1, \\ 1 \le n_E &\le 2^h, \\ h \le n_I &\le 2^h-1, \\ \lceil \log_2(n+1) \rceil - 1 \le h &\le n-1. \end{aligned}$$

A chain attains $n = h+1$; a perfect tree attains $n = 2^{h+1}-1$. The ceiling accounts for integer height and strengthens the real lower bound $\log_2(n+1) - 1$.

3.3 Full binary tree

Each internal node contributes two child edges:

$$2n_I = n - 1 = n_I + n_E - 1 \quad \Longrightarrow \quad n_E = n_I + 1.$$

Thus n is odd, $n_E = (n+1)/2$, and $n_I = (n-1)/2$.

$$\begin{aligned} 2h+1 \le n &\le 2^{h+1}-1, \\ h+1 \le n_E &\le 2^h, \\ h \le n_I &\le 2^h-1, \\ \lceil \log_2(n+1) \rceil - 1 \le h &\le (n-1)/2. \end{aligned}$$

A full tree can still have height $\Theta(n)$: a long internal spine with a leaf sibling at each internal node.

3.4 Perfect and complete trees

For a perfect tree,

$$n = 2^{h+1}-1, \quad n_E = 2^h, \quad n_I = 2^h - 1, \quad h = \log_2(n+1) - 1.$$

For a complete tree,

$$2^h \le n \le 2^{h+1}-1, \qquad h = \lfloor \log_2 n \rfloor.$$

The height formula follows from $h \le \log_2 n < h+1$. From its array layout, a complete tree also has $n_I = \lfloor n/2 \rfloor$ and $n_E = \lceil n/2 \rceil$.

Example: with $h = 3$, a binary tree has 4–15 nodes; a full tree 7–15; a complete tree 8–15; a perfect tree exactly 15 nodes and 8 leaves.

4 Tree ADTs

Source: PDF pages 35–36. General tree accessors include root, parent, and children. Queries test whether a position is root, internal, or external. Other operations report size, iterate elements/positions, replace an element, or swap elements. Updates depend on the application.

A binary-tree ADT adds left-child, right-child, and sibling accessors and specialized updates such as expanding a leaf or removing a leaf together with its parent.

5 Traversal orders

Source: PDF pages 37–59,65–68. A traversal visits every node in a specified order; process ordered children left to right.

Traversal	Binary-tree order
Breadth-first	Level by level, using a queue
Preorder	Root, left subtree, right subtree
Inorder	Left subtree, root, right subtree
Postorder	Left subtree, right subtree, root

Preorder and postorder apply to arbitrary rooted trees; the lecture's inorder is for binary trees.

5.1 Breadth-first traversal

```
BFS(root):
    if root == null: return
    enqueue(root)
    while queue is not empty:
        v = dequeue()
        visit(v)
        for each child w of v, in order:
            enqueue(w)
```

Each node is enqueued and dequeued once. Time is $\Theta(n)$ for constant-time visits; auxiliary space is $O(w)$ for maximum level width w, hence $O(n)$ worst case and $O(1)$ for a chain.

5.2 Recursive depth-first traversal

```
preorder(v):
    if v == null: return
    visit(v)
    preorder(v.left)
    preorder(v.right)

inorder(v):
    if v == null: return
    inorder(v.left)
    visit(v)
    inorder(v.right)

postorder(v):
    if v == null: return
    postorder(v.left)
    postorder(v.right)
    visit(v)
```

Null checks correctly handle missing children. Time is $\Theta(n)$; auxiliary recursion space is $O(h+1)$, which is $O(\log n)$ for a complete tree and $O(n)$ for a chain. Output storage is separate.

Lecture example (PDF page 46): A has children B and G; B has children C and D; D has children E and F; G has children H and I.

- BFS: A, B, G, C, D, H, I, E, F.

- Preorder: A, B, C, D, E, F, G, H, I.
- Inorder: C, B, E, D, F, A, H, G, I.
- Postorder: C, E, F, D, B, H, I, G, A.

5.3 Iterative preorder and Euler tour

```
if root != null: push(root)
while stack is not empty:
    v = pop()
    visit(v)
    if v.right != null: push(v.right)
    if v.left != null: push(v.left)
```

Push the right child first so the left child is popped first. The pending-node stack uses $O(h+1)$ space in a binary tree.

An Euler tour offers three action points: before traversing the left subtree (preorder), between left and right (inorder), and after the right subtree (postorder). This unifies the three depth-first orders.

6 Traversal applications

Source: PDF pages 42–43,60–64.

6.1 Structured documents and directory sizes

Preorder prints headings before subheadings, with depth determining indentation. Postorder combines child sizes before calculating a directory total. The slide's files $10, 25, 20, 3, 2, 1$ KB total 61 KB, ignoring directory overhead.

6.2 Subtree height

Let a missing subtree have height -1, giving leaves height zero:

```
height(v):
    if v == null: return -1
    return 1 + max(height(v.left), height(v.right))
```

This postorder algorithm takes $\Theta(n)$ time for the queried subtree and $O(h+1)$ auxiliary space.

6.3 Expression trees

Leaves are operands; internal nodes are binary operators. Preorder gives prefix notation, postorder postfix. Plain inorder can lose grouping; print parentheses around internal subtrees to preserve the expression.

```
printExpr(v):
    if v is a leaf: print(v.element); return
    print("(")
    printExpr(v.left)
    print(v.operator)
    printExpr(v.right)
    print(")")

evalExpr(v):
    if v is a leaf: return its numeric value
    x = evalExpr(v.left)
    y = evalExpr(v.right)
    return apply(v.operator, x, y)
```

For $((2 \times (a-1)) + (3 \times b))$, postfix is `2 a 1 - * 3 b * +`. With $a = 5, b = 2$, the value is 14. Time is $\Theta(n)$ with constant-time operator evaluation or output per token.

7 Representations and updates

Source: PDF pages 70–81.

7.1 Linked binary tree

Each node stores an element, parent, left child, and right child. Given valid nodes, local access, replacement, and element swapping cost $O(1)$. Swapping elements does not exchange subtrees.

`expandExternal(v)` adds two children to a leaf and increases size by two. Set each child's parent to v.

The slide's `removeExternal(v)` removes leaf v and its parent p, replacing p by sibling s (also called removal above an external node):

```
p = v.parent
s = sibling(v)
if p is root:
    root = s
    s.parent = null
else:
    g = p.parent
    replace g's child reference to p with s
    s.parent = g
size = size - 2
invalidate v and p
```

Assume a full tree and a non-root leaf with a sibling. Updating `s.parent` is necessary in the non-root case. This local update costs $O(1)$.

7.2 Array binary tree

With zero-based root numbering:

$$\text{left}(i) = 2i + 1, \quad \text{right}(i) = 2i + 2, \quad \text{parent}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor \ (i > 0).$$

A child exists only if its index is within storage and its slot is occupied. In a complete tree stored in $[0, n)$, child indices must be strictly less than n. Node count alone does not identify occupied indices in a sparse tree.

A complete tree uses $O(n)$ consecutive slots. A right-only chain of n nodes ends at index $2^n - 2$, requiring array length $2^n - 1$. Moving a subtree can require relocating descendants; linked representations avoid these holes and permit local rewiring.

7.3 General tree and binary encoding

A general linked node stores an element, parent, and child-reference collection. An ordered general tree can be encoded by binary links:

- left link: first child in the general tree;
- right link: next sibling in the general tree.

The roots agree, and the binary root has no next sibling. Binary preorder preserves general-tree preorder. Because right now means sibling, binary depth, height, and parent relations must not be read as those of the original hierarchy.

Study checklist Classify full/perfect/complete trees; derive leaf/height bounds; produce four traversal orders; justify queue and recursion space; preserve parent references when rewiring; explain why complete trees fit arrays better than sparse trees.